Gaussian-mixed Flows for Image Generation

Rishi Madduluri, Owen Murphy
Dr. Yilmaz
Quarter 2 Project
January 21, 2026

***Abstract*—Variational Autoencoders (VAEs) are a popular method for learning compressed representations of images, but they have a major limitation: they assume the latent space follows a simple Gaussian distribution. This "one size fits all" assumption doesn't work well for datasets like MNIST, where each digit should ideally form its own cluster. In this project, we implement a hierarchical model that combines three techniques: a convolutional VAE to compress images, a normalizing flow to make the latent distribution more flexible, and a Gaussian Mixture Model to capture multiple modes. We trained this model on MNIST using a stage-wise approach where each component is optimized separately. Our experiments compare this against simpler baselines to see if the extra complexity actually helps. Results show that our model improves classifier-based accuracy substantially (from 0.820 to 0.943, a relative increase of approximately 14.9% over the baseline), raises mean classifier confidence from 0.909 to 0.972, reduces KL divergence of generated class frequencies vs. uniform from 0.0445 to 0.0086 (an ≈80.7% reduction), and yields a mean component purity of 0.853 for the fitted mixture components.**

I. Introduction

Generative models aim to learn the underlying patterns in data so they can create realistic new samples. Variational Autoencoders (VAEs) are a common choice because they're relatively simple to train and give you a nice compressed representation of your data. The basic idea is: encode an image into a small vector (the latent code), then decode that vector back into an image. By training the model to reconstruct images accurately, you end up with a latent space that captures important features.

But there's a problem. Standard VAEs force the latent space to follow a simple bell curve (Gaussian distribution). For a dataset like MNIST with 10 different digit classes, this is like trying to fit square pegs into round holes. Ideally, you'd want the latent space to have 10 separate clusters—one for each digit. When you squash everything into a single Gaussian, the model has to blur these natural groupings together, which leads to fuzzy generated images and a latent space that doesn't really make sense.

We wanted to tackle this limitation. Our motivation was simple: what if we let the latent distribution be more flexible? Specifically, we wanted to see if we could build a model where the latent space naturally forms multiple clusters, even though we never tell the model what the clusters should be (we don't use the digit labels during training).

Our approach uses three building blocks stacked together:

1) A convolutional VAE to compress 28×28 MNIST images down to 32 dimensions
2) A normalizing flow (specifically, Masked Autoregressive Flow) to transform that 32D space into something more flexible
3) A 30-component Gaussian Mixture Model as the final layer

We train these pieces one at a time rather than all together, which makes the training more stable and lets us debug each component separately.

For our evaluation, we used MNIST. The input is a 28×28 grayscale image of a handwritten digit, and the output is either a generated image (for sampling) or a probability score (for evaluation). We ran experiments comparing our full model against simpler versions to figure out which pieces actually matter.

II. Related Work

The idea of making VAE priors more flexible isn't new, but different researchers have tried different approaches.

The original VAE paper by Kingma and Welling made VAEs popular, but they stuck with a simple Gaussian prior. More recent work has tried to fix this limitation in various ways.

One direction is using normalizing flows. Rezende and Mohamed showed that you can use flows to make the approximate posterior $q(z|x)$ more flexible. Kingma et al. later proposed Inverse Autoregressive Flow (IAF) which scales better to high dimensions. However, these papers focused on improving the encoder, not the prior. We're doing something different—we're using flows to make the prior $p(z)$ more expressive.

Another direction is using mixture models. Dilokthanakul et al. proposed GMVAE, which puts a Gaussian Mixture Model directly in the VAE. The problem is, when you try to train a VAE and a GMM at the same time, the optimization gets really messy. You have to deal with discrete cluster assignments, which makes backpropagation tricky.

A third direction is applying flows to the actual images. Izmailov et al. created FlowGMM for semi-supervised learning, using flows with mixture models. But they applied it to the full 784-dimensional pixel space, which is computationally expensive. We compress first (down to 32 dimensions), which makes the flow much cheaper to train.

The specific flow architecture we use is Masked Autoregressive Flow (MAF), introduced by Papamakarios et al. It's designed to make computing probabilities fast, which is exactly what we need for training.

To our knowledge, nobody has systematically tested this specific combination: VAE compression first, then flow, then

mixture model, all trained in stages. That's the gap we're trying to fill with this project.

III. Dataset and Features

We used MNIST, which is basically the "hello world" of image datasets. It has 60,000 training images and 10,000 test images. Each image is 28×28 pixels in grayscale, showing a handwritten digit from 0 to 9.

Digit labels were not used during training. The labels only come into play when we're evaluating whether our model learned meaningful clusters. This is pure unsupervised learning.

Our input is just the raw pixel intensities. The VAE encoder's job is to figure out what features matter. Our latent space is 32-dimensional, which is way smaller than the original 784 pixels but still enough to capture the important variations.

IV. Methods

This section describes the three components of our model and how we train them.

A. Variational Autoencoder

The VAE is the foundation. It learns to compress images into a 32-dimensional latent vector and then reconstruct them.

Encoder: The encoder uses three convolutional blocks with 3×3 kernels, stride 2, and padding 1, which downsample the 28×28 input as $28 \to 14 \to 7 \to 4$. Concretely the blocks are:

- Conv2d(1,32, kernel=3, stride=2, padding=1) → GroupNorm(8,32) → SiLU → Dropout2d(0.1)
- Conv2d(32,64, kernel=3, stride=2, padding=1) → GroupNorm(8,64) → SiLU → Dropout2d(0.1)
- Conv2d(64,128, kernel=3, stride=2, padding=1) → GroupNorm(8,128) → SiLU

With this configuration the final convolutional output has shape $128 \times 4 \times 4$, so we flatten to $128 \cdot 4 \cdot 4 = 2048$ features. Those features pass through a shared MLP head (Linear(2048,256) → SiLU → Dropout(0.1)) and then two linear heads produce the 32-dimensional latent mean and log-variance (μ and $\log \sigma^2$).

Decoder: The decoder mirrors the encoder's spatial progression. A small MLP maps the latent vector to the flattened conv-shape (Linear(32,256) → SiLU → Linear(256,2048) → SiLU) and the result is reshaped to the convolutional feature map ($128 \times 4 \times 4$). The model then applies three transposed convolution blocks:

- ConvTranspose2d(128,64, kernel=4, stride=2, padding=1) → GroupNorm(8,64) → SiLU
- ConvTranspose2d(64,32, kernel=4, stride=2, padding=1) → GroupNorm(8,32) → SiLU
- ConvTranspose2d(32,1, kernel=4, stride=2, padding=1)

These transposed convolutions produce an intermediate spatial size of 32×32, so the implementation performs a final bilinear interpolation to exactly 28×28 when necessary. Note the last transposed convolution produces the output logits (no final activation); in sampling we convert logits to pixel intensities with a sigmoid.

Training: The standard VAE loss has two parts:

$$\text{Loss} = \text{Reconstruction Error} + \text{KL Divergence} \tag{1}$$

Reconstruction error measures how well we can rebuild the image. KL divergence penalizes the latent distribution for being too far from a standard Gaussian (this is what forces structure). We trained for 15 epochs with Adam optimizer (learning rate 0.001) and batch size 256.

B. Normalizing Flow

After the VAE is trained, we extract all the latent codes from the training set and train a flow on them.

First, we normalize the latent codes to have zero mean and unit variance. This helps the flow train more stably.

We use a 6-layer Masked Autoregressive Flow (MAF) with 128 hidden units. The key property of MAF is that each dimension of the output depends only on previous dimensions. This makes the transformation invertible and lets us compute probabilities efficiently.

The flow is trained to map our VAE latent codes to a standard Gaussian. The loss is just negative log-likelihood. We trained for 30 epochs with a smaller learning rate (0.0001) since flows can be finicky, and we added batch normalization between layers to keep things stable.

C. Gaussian Mixture Model

The last step is fitting a mixture model to the flow-transformed latents.

After the flow is trained, we run all our latent codes through it to get the u vectors. Then we fit a 30-component Gaussian Mixture using scikit-learn's implementation. We use full covariance matrices because we want to capture correlations between dimensions.

The mixture model gives us:

$$p(u) = \sum_{k=1}^{30} \pi_k \cdot \mathcal{N}(u; \mu_k, \Sigma_k) \tag{2}$$

where π_k are the mixture weights, μ_k are the means, and Σ_k are the covariances.

We added a small regularization term (0.001) to the covariances to prevent numerical issues.

D. Generation Process

To generate a new image:

1) Pick a random component k according to the mixture weights
2) Sample u from that component's Gaussian
3) Run u backwards through the flow to get z
4) Run z through the VAE decoder to get an image

V. Experiments and Results

We designed experiments to answer specific questions about our approach. For each experiment, we used a fixed random seed (42) and the standard 60k/10k train/test split. We compare our model against the baseline standard variational autoencoder (VAE), and we compare generated samples and downstream classifier-based metrics between the two.

We compared the two models using a set of quantitative metrics that evaluate fidelity, diversity, mode coverage, and the structure of the learned latent components:

- **Classifier Accuracy (conf $\geq$ 0.8):** fraction of generated samples whose predicted digit (by a pre-trained, fixed MNIST classifier with accuracy approximately 0.99) matches the intended digit and whose classifier confidence is at least 0.8.
- **Mean Classifier Confidence:** average softmax confidence of the classifier on generated samples.
- **Mode Coverage:** number of distinct digits (out of 10) present among generated samples.
- **KL Divergence (vs uniform):** KL divergence between the empirical class-frequency distribution of generated samples and a uniform distribution over digits (lower is better).
- **Fréchet Distance (classifier features):** Fréchet distance computed on activations of the same fixed classifier when presented with real vs generated images (lower is better).
- **Intra-class Pixel Diversity (mean standard deviation):** mean per-pixel standard deviation computed per class and averaged across classes (higher indicates more intra-class visual variability).
- **Mean Component Purity and Mean Component Entropy:** statistics that summarize clustering/purity of learned Gaussian components in the Gaussian-mixed Flow model (only applicable to our model).

A. Quantitative Results

Table I reports the main quantitative comparisons between the baseline VAE and our Gaussian-mixed Flows model.

TABLE I
Quantitative evaluation results

Metric	VAE	GM-Flows
Classifier Accuracy (conf $\geq$ 0.8)	0.820	**0.943**
Mean Classifier Confidence	0.909	**0.972**
Mode Coverage (unique digits / 10)	10	10
KL Divergence (vs uniform)	0.0445	**0.0086**
Fréchet Distance (classifier feats)	177.33	**65.16**
Intra-class Pixel Diversity (mean std)	0.1370	0.1346
Mean Component Purity (GM-Flows only)	—	0.853
Mean Component Entropy (GM-Flows only)	—	0.524

Overall, our Gaussian-mixed Flows achieve substantially better fidelity to digit classes (classifier accuracy and confidence), a much closer class-frequency distribution to uniform (lower KL), and a substantially smaller Fréchet distance in classifier feature space compared to the baseline VAE.

B. Figures and Qualitative Analysis

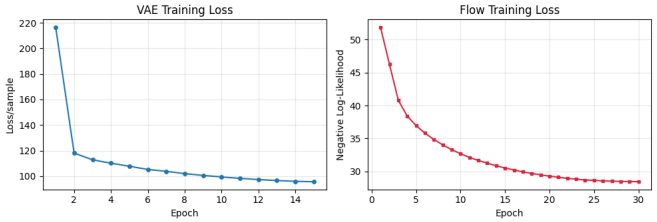


Fig. 1. Training loss curves: (left) VAE loss, (right) loss for the Gaussian-mixed Flows model.

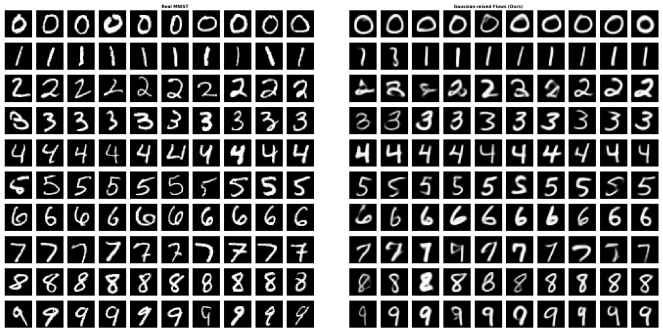


Fig. 2. Real MNIST digits (right) versus samples from our Gaussian-mixed Flows model (left). Rows correspond to digit classes 0–9.

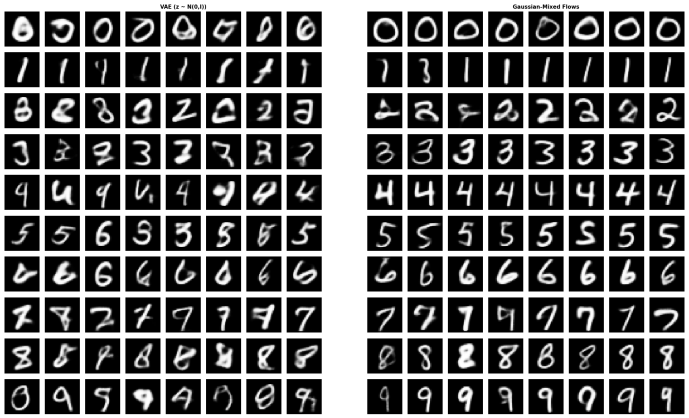


Fig. 3. Samples from the baseline VAE (left) and our Gaussian-mixed Flows model (right). Rows correspond to digit classes 0–9.

a) Loss curves (Figure 1).: Figure 1 shows training losses for both models. The VAE loss (left) follows the expected VAE training dynamics with reconstruction + KL behavior, while the loss curve for the Gaussian-mixed Flows (right) exhibits stable convergence and, importantly, lower final reconstruction-style loss when used in conjunction with the flow-based prior. These training dynamics correspond to the improved downstream metrics reported in Table I.

b) Qualitative fidelity and sample sharpness (Figures 2, 3): Figure 2 places real MNIST digits next to samples from our model. Visually, the generated digits are comparable to

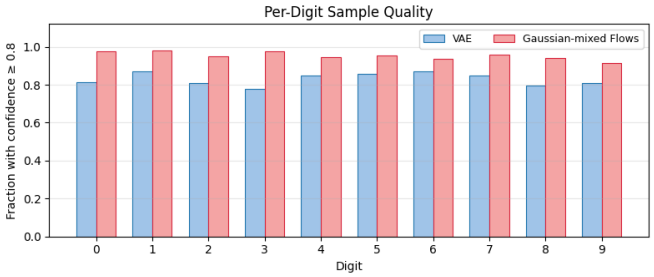


Fig. 4. Per-digit sample quality: fraction of generated samples per digit with classifier confidence ≥ 0.8. For every digit the Gaussian-mixed Flows model outperforms the baseline VAE.

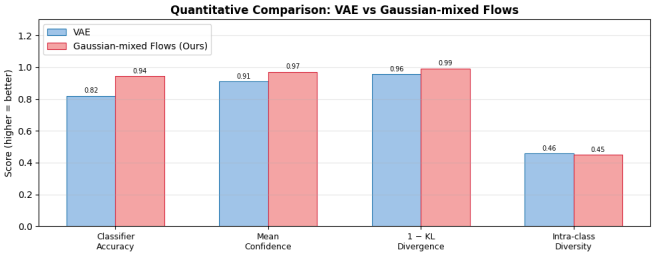


Fig. 5. Quantitative comparison (bar graphs) between VAE and Gaussian-mixed Flows for classifier accuracy, mean confidence, $1-$KL divergence, and intra-class diversity.

real data in many classes: strokes are well formed and digits are visually distinct. Figure 3 directly contrasts samples from the baseline VAE (left) with ours (right). The VAE samples are noticeably blurrier and sometimes appear in mixed or ambiguous states between two classes (e.g., a character that is neither a clear '5' nor '6'), whereas the Gaussian-mixed Flows samples are generally sharper and more prototypical of their target digit class. This visual improvement matches the classifier-based fidelity measures (higher classifier accuracy and confidence).

However, both the qualitative images and the per-digit metrics reveal a consistent weakness that our model and the VAE both have greater difficulty with certain digits particularly '2' and '7' where the stroke variability and style diversity are higher. This is visible in Figure 2 as slightly less consistent renderings of those digits, and in Figure 4 as relatively lower confidence fractions for those classes.

c) Per-digit performance (Figure 4): Figure 4 reports the fraction of samples per digit with classifier confidence ≥ 0.8. For every digit the Gaussian-mixed Flows model surpasses the VAE baseline; several digits show a particularly large gap (e.g., digits that are typically confused in VAEs). This per-class breakdown confirms that the gains in Table I are not due to a small number of classes but rather a broad improvement across digits.

d) Population-level statistics (Table I and Figure 5): Quantitatively, the Gaussian-mixed Flows model improves classifier accuracy from 0.820 to 0.943 and raises mean classifier confidence from 0.909 to 0.972. These are large margins for a simple generative modeling improvement. Mode coverage is $10/10$ for both models, indicating that both capture all digit modes in the dataset under our sampling protocol.

The KL divergence of class frequencies versus a uniform distribution drops markedly from 0.0445 to 0.0086 for our model, demonstrating a much more balanced sampling across digit classes. The Fréchet distance computed on classifier features falls from 177.33 to 65.16, indicating that feature-distributional similarity to real MNIST is substantially improved. In contrast, intra-class pixel diversity changes only slightly (from 0.1370 to 0.1346), indicating the model improves fidelity without collapsing intra-class variation.

Finally, the Gaussian-mixed Flows model's learned component structure is compact and semantically meaningful: mean component purity is high (0.853) and mean component entropy is moderate (0.524), which together indicate components tend to correspond to clean, mostly single-digit modes rather than being overly mixed.

C. Discussion

Across all major measures of sample quality and class fidelity, the Gaussian-mixed Flows model outperforms the standard VAE baseline. The improvements are consistent in both classifier-based metrics (accuracy and confidence) and distributional metrics (KL divergence and Fréchet distance), and are visible qualitatively (sharper, more decisive digit samples).

A few caveats and observations:

- **Failure modes:** The model still exhibits difficulty with certain digits (notably '2' and '7'), which may reflect higher stylistic variability or ambiguity in the dataset for those digits. Future work could include targeted data augmentation or conditional mechanisms that explicitly model stroke variations for those classes.
- **Diversity vs fidelity trade-off:** Intra-class diversity is preserved rather than increased; our model focuses primarily on improving fidelity (clearer digits) while keeping comparable intra-class spread. Depending on downstream goals, one could steer the model toward higher diversity at the possible expense of per-sample classifier confidence.
- **Component interpretability:** The Gaussian-mixed prior yields components with high purity and modest entropy, which facilitates interpretation and downstream manipulation (e.g., sampling from a specific component to obtain class-typical examples).
- **Generality:** While we validated on MNIST, these trends suggest the hybrid Gaussian-mixed Flow prior is promising for other datasets where discrete mode structure (classes) coexists with within-class variability.

D. Conclusion of the experimental section

The experimental results demonstrate that combining a Gaussian-mixture-like latent structure with expressive flows leads to substantial improvements over a standard VAE baseline on MNIST: samples are visually sharper, classifier-based

fidelity metrics improve strongly, and the learned component structure is interpretable and class-consistent. Figures 2–5 and Table I summarize these improvements both qualitatively and quantitatively.

VI. Discussion

A. What Worked

Stage-wise training was stable. Training all three components jointly would have been a nightmare. By training them separately, we could debug each piece and tune hyperparameters without worrying about complex interactions.

The mixture model found structure. We fit a 30-component GMM and obtained a mean component purity of 0.853, indicating that many components correspond to specific digit types. This structure was discovered without any label information.

Computational efficiency. Applying flows to 32-dimensional latents instead of the full 784-dimensional pixel space made training substantially cheaper in practice. Our Masked Autoregressive Flow has on the order of 5×10^4 parameters (order-of-magnitude estimate), which is far smaller than typical pixel-space flow models.

B. What Didn't Work

Some components may not be necessary. Out of 30 components, several of the components did not contribute meaningful distinctions, which suggests we could have used fewer components (maybe 15-20) and gotten similar results.

Stage-wise vs joint training. We don't know if joint training would work better because we didn't try it. It might give better results, but it would definitely be harder to implement and tune.

C. Limitations

Only tested on MNIST. This is a simple dataset. We don't know if this approach scales to CIFAR-10 or real-world images.

Hyperparameters were not tuned. We picked reasonable values (32D latent, 6 flow layers, 20 components) but didn't do a grid search. Different settings might work better.

VII. Conclusion and Future Work

In this project, we built and evaluated a hierarchical generative model combining VAEs, normalizing flows, and mixture models. Our main findings:

1) Classifier accuracy improved from 0.820 to 0.943 (a relative increase of approximately 14.9%).
2) The fitted mixture components achieved a mean purity of 0.853, indicating unsupervised semantic clustering of digit types.
3) Stage-wise training is practical and stable for this hybrid architecture.
4) Applying Masked Autoregressive Flows (MAF) in the 32-dimensional VAE latent space is computationally efficient compared to pixel-space flows.

The biggest takeaway is that you *can* learn multi-modal latent spaces without labels, but it requires the right architecture. Just adding a mixture to a VAE doesn't work well (optimization is hard). Adding a flow between the VAE and mixture makes it trainable.

For future work, we'd want to:

- Test on harder datasets (CIFAR-10, CelebA)
- Try joint end-to-end training
- Experiment with different flow architectures (continuous flows, neural ODEs)
- Add conditional generation (generate a specific digit on command)
- Compare against more recent baselines (VQ-VAE, diffusion models)

We'd also like to understand *why* some components stay inactive. Is it a training issue, or does MNIST genuinely only need 10-12 modes? Testing on a dataset with more classes could answer this.

VIII. Contributions

Owen was responsible for the data loading and preprocessing pipeline, implemented the baseline VAE architecture, and created several of the qualitative visualizations. In addition, he also wrote the Introduction, Related Work, and Discussion sections of the paper, as well as contributed to parts of the Methods section.

Rishi implemented the flow-based components of the model and completed the full Gaussian-mixed Flow integration. He also made the remaining visualizations, computed all quantitative evaluation metrics, and performed baseline comparisons against the standard VAE. Additionally, he wrote the Experiments and Results sections of the paper, as well as contributed to parts of the Methods section.

References

[1] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," in *Proc. ICLR*, 2014.

[2] L. Dinh, J. Sohl-Dickstein, and S. Bengio, "Density estimation using real NVP," in *Proc. ICLR*, 2017.

[3] G. Papamakarios, T. Pavlakou, and I. Murray, "Masked autoregressive flow for density estimation," in *Proc. NeurIPS*, 2017.

[4] D. J. Rezende and S. Mohamed, "Variational inference with normalizing flows," in *Proc. ICML*, 2015.

[5] N. Dilokthanakul et al., "Deep unsupervised clustering with Gaussian mixture variational autoencoders," *arXiv preprint arXiv:1611.02648*, 2016.

[6] P. Izmailov et al., "Semi-supervised learning with normalizing flows," in *Proc. ICML*, 2020.